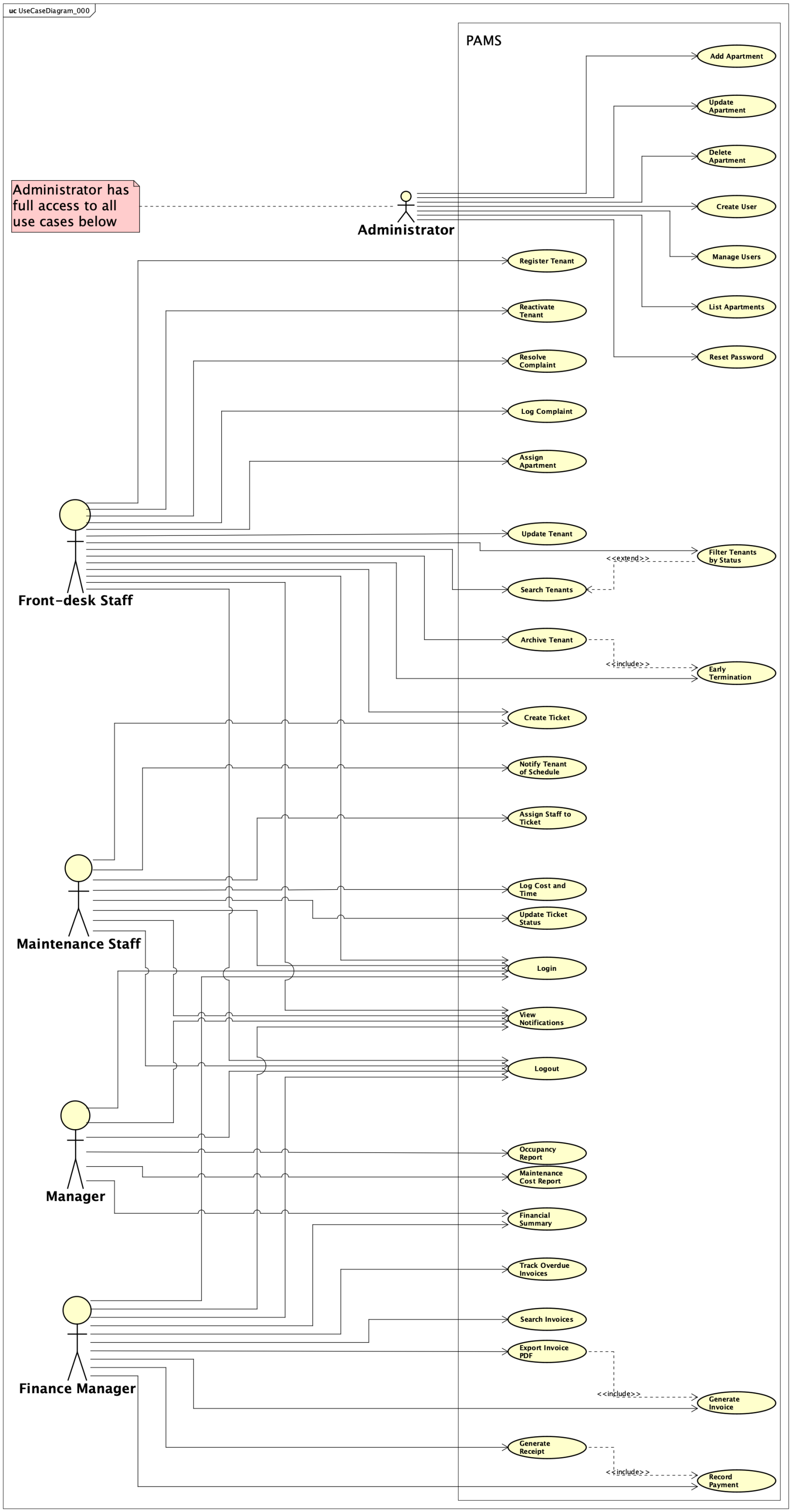
UML Diagrams

Note on formatting: All diagrams have been exported on A3 pages to preserve readability given their size and detail. If any element appears small at default zoom, zooming in will render it clearly without loss of quality.

Author: Omar Al Frouh

Student ID: 24020879

USE-CASE DIAGRAM



CLASS DIAGRAM

pkg

Entities

User
+ id : int
+ username : String
+ password_hash : String
+ role : String
+ created_at : String
+ last_login : String

Staff
+ user_id : int
+ employee_id : String
+ department : String

Tenant
+ user_id : int
+ ni_number : String
+ full_name : String
+ phone_number : String
+ email : String
+ occupation : String
+ references_info : String
+ emergency_contact : String
+ apartment_requirements : String
+ apartment_id : int
+ lease_start_date : String
+ lease_end_date : String
+ deposit_amount : float
+ monthly_rent : float
+ status : String

Apartment
+ id : int
+ unit_number : String
+ address : String
+ city : String
+ number_of_rooms : int
+ rental_price : float
+ square_footage : int
+ status : String

MaintenanceRequest
+ id : int
+ apartment_id : int
+ tenant_id : int
+ staff_id : int
+ description : String
+ priority : String
+ status : String
+ date_reported : String
+ scheduled_date : String
+ date_resolved : String
+ time_spent : float
+ cost : float

Invoice
+ id : int
+ tenant_id : int
+ issue_date : String
+ due_date : String
+ total_amount : float
+ status : String

Payment
+ id : int
+ invoice_id : int
+ amount : float
+ payment_date : String
+ method : String
+ transaction_ref : String

Complaint
+ id : int
+ tenant_id : int
+ date_logged : String
+ subject : String
+ description : String
+ status : String
+ resolved_date : String

Utilities

<<utility>> pdf_generator
+ generate_invoice_pdf(data : dict, out_path : String) : void
+ generate_receipt_pdf(data : dict, out_path : String) : void

<<utility>> ui_helpers
+ create_card(parent : Widget) : Frame
+ create_kpi_tile(parent : Widget, label : String, value : String) : Frame
+ show_success(root : Widget, msg : String) : void
+ show_error(root : Widget, msg : String) : void
+ show_warning(root : Widget, msg : String) : void
+ show_info(root : Widget, msg : String) : void
+ confirm_dialog(root : Widget, title : String, msg : String, danger : bool) : bool
+ create_badge(parent : Widget, text : String, color : String) : Label
+ create_empty_state(parent : Widget, icon : String, msg : String) : Frame
+ setup_treeview_scroll(tree : Treeview) : void
+ autofit_columns(tree : Treeview) : void
+ enable_trackpad_scroll(frame : Widget) : void

<<utility>> theme
+ BG : String
+ SURFACE : String
+ TEXT : String
+ PRIMARY : String
+ SUCCESS : String
+ WARNING : String
+ DANGER : String
+ FONT_BODY : tuple
+ FONT_H2 : tuple
+ SP_S : int
+ SP_L : int
+ RADIUS_SM : int
+ RADIUS_LG : int
+ SIDEBAR_WIDTH : int
+ INPUT_HEIGHT : int

Views

PaymentView
+ build() : void

InvoiceView
+ build() : void

MainView
- current_user : tuple
- managers : dict
+ setup_sidebar() : void
+ build_header() : void
+ show_notifications() : void

ReportView
+ build() : void

TenantView
+ build() : void

MaintenanceView
+ build() : void

ApartmentView
+ build() : void

LoginView
- auth_manager : AuthManager
- on_login_success : callable
+ handle_login() : void

UserView
+ build() : void

Managers

PaymentManager
- db_manager : DatabaseManager
+ record_payment(invoice_id : int, amount : float, method : String, transaction_ref : String) : int
+ get_all_payments() : list
+ get_financial_summary() : dict
+ get_collected_vs_pending() : dict
+ get_payment_history_by_method() : list
+ get_monthly_revenue() : list
+ void_payment(payment_id : int) : void
+ get_payments_for_tenant(tenant_id : int) : list
+ get_receipt_data(payment_id : int) : dict

InvoiceManager
- db_manager : DatabaseManager
+ generate_invoice(tenant_id : int, amount : float) : int
+ get_all_invoices() : list
+ get_overdue_invoices() : list
+ mark_paid(invoice_id : int) : void
+ delete_invoice(invoice_id : int) : void
+ get_invoices_for_tenant(tenant_id : int) : list
+ get_payments_for_invoice(invoice_id : int) : list
+ get_invoice_pdf_data(invoice_id : int) : dict
+ search_invoices(status : String, tenant_name : String) : list
+ get_overdue_with_details() : list
+ get_invoice_aging() : dict

NotificationManager
- db_manager : DatabaseManager
+ get_overdue_invoices() : list
+ get_upcoming_maintenance() : list
+ get_expiring_leases(days : int) : list
+ get_notification_count() : int
+ get_notifications() : list

TenantManager
- db_manager : DatabaseManager
+ register_tenant(username : String, password : String, ni_number : String, full_name : String, phone : String, email : String, occupation : String, references : String, emergency_contact : String, apt_requirements : String) : bool
+ get_all_tenants() : list
+ get_archived_tenants() : list
+ assign_apartment(tenant_user_id : int, apartment_id : int, lease_start : String, lease_end : String, monthly_rent : float, deposit : float) : bool
+ reactivate_tenant(tenant_user_id : int) : bool
+ archive_tenant(tenant_user_id : int) : bool
+ search_tenants(query : String) : list
+ filter_tenants_by_status(status : String) : list
+ get_tenant_count_by_status() : dict
+ early_termination(tenant_user_id : int) : dict
+ get_tenant_by_id(tenant_user_id : int) : tuple
+ update_tenant(tenant_user_id : int, full_name : String, phone : String, email : String, occupation : String, references : String, emergency_contact : String, apt_requirements : String) : bool
+ log_complaint(tenant_id : int, subject : String, description : String) : bool
+ get_all_complaints() : list
+ get_complaints_for_tenant(tenant_id : int) : list
+ resolve_complaint(complaint_id : int) : bool

ReportManager
- db_manager : DatabaseManager
+ get_occupancy_report(city : String) : list
+ get_occupancy_summary(city : String) : dict
+ get_financial_summary() : dict
+ get_financial_breakdown() : dict
+ get_maintenance_cost_report(city : String) : list
+ get_maintenance_cost_summary() : dict
+ get_cities() : list

MaintenanceManager
- db_manager : DatabaseManager
+ create_ticket(apartment_id : int, tenant_id : int, description : String, priority : String) : bool
+ get_all_tickets() : list
+ assign_staff(ticket_id : int, staff_user_id : int, scheduled_date : String) : bool
+ update_status(ticket_id : int, new_status : String) : bool
+ log_costs(ticket_id : int, time_spent : float, cost : float) : bool
+ get_available_staff() : list
+ get_maintenance_costs_summary() : dict
+ get_cost_report_by_apartment() : list
+ get_cost_report_by_city() : list
+ get_ticket_detail(ticket_id : int) : dict
+ get_tickets_for_tenant(tenant_id : int) : list
+ delete_ticket(ticket_id : int) : void
+ get_tickets_by_status_count() : dict
+ notify_tenant_scheduled_date(ticket_id : int, scheduled_date : String) : dict

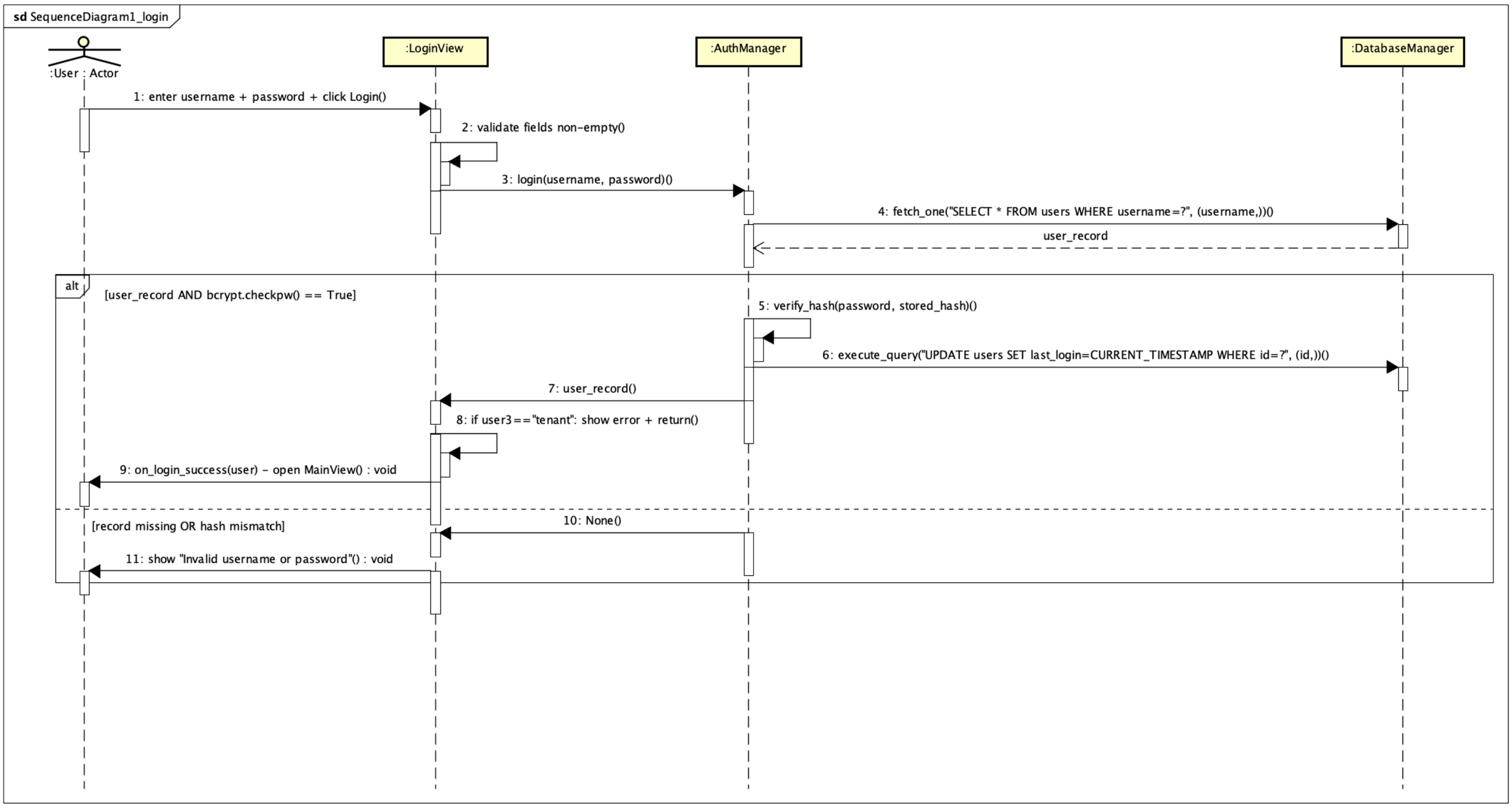
PropertyManager
- db_manager : DatabaseManager
+ add_apartment(unit_number : String, address : String, rental_price : float, square_footage : int, city : String, number_of_rooms : int) : bool
+ get_all_apartments() : list
+ get_vacant_apartments() : list
+ get_cities() : list
+ update_apartment(apt_id : int, address : String, city : String, rooms : int, rent : float, sqft : int) : void
+ delete_apartment(apt_id : int) : void
+ get_apartment_detail(apt_id : int) : dict
+ update_status(apartment_id : int, status : String) : void

AuthManager
- db_manager : DatabaseManager
+ login(username : String, password : String) : tuple
+ verify_hash(plain : String, hashed : String) : bool
+ create_user(username : String, password : String, role : String, employee_id : String, department : String) : bool
+ get_all_users() : list
+ reset_password(user_id : int, new_password : String) : void
+ update_user(user_id : int, role : String, employee_id : String, department : String, current_user_id : int) : void
+ delete_user(user_id : int, current_user_id : int) : void
+ get_dashboard_stats() : dict

Core

DatabaseManager
- db_name : String
+ get_connection() : Connection
+ execute_query(query : String, params : tuple) : Cursor
+ fetch_all(query : String, params : tuple) : list
+ fetch_one(query : String, params : tuple) : tuple
+ create_tables() : void
- _apply_migrations() : void
- _migrate_drop_apt_unique() : void

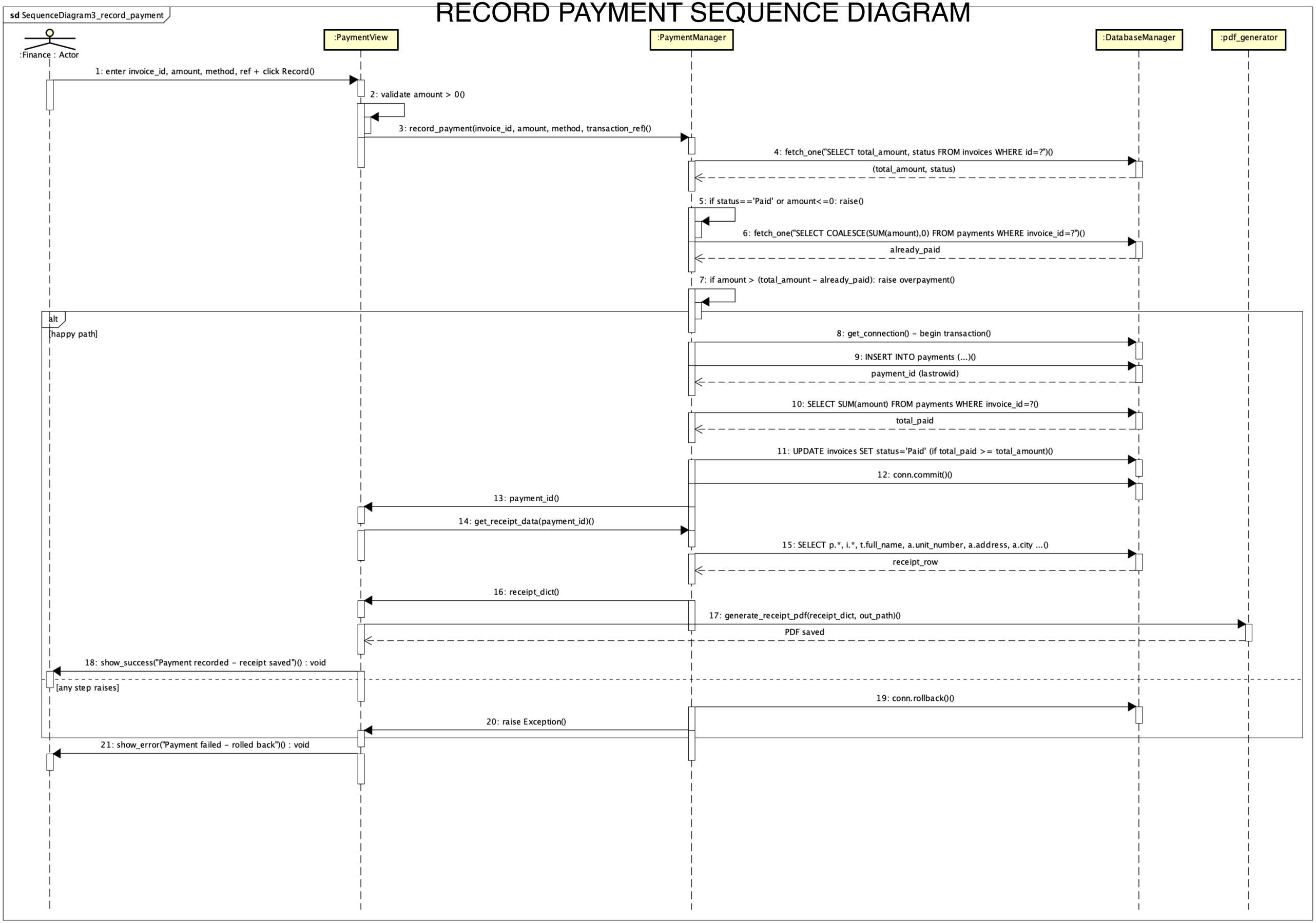
LOGIN SEQUENCE DIAGRAM



TENANT SEQUENCE DIAGRAM



RECORD PAYMENT SEQUENCE DIAGRAM



EARLY TERMINATION SEQUENCE DIAGRAM

